```
void enkucuksil(struct hucre **liste)//listenin basinda olabilir o yüzden **liste
{
 struct hucre * enk=NULL;
 struct hucre * onceki=NULL;
 struct hucre * enkucuktenonceki=NULL;
 struct hucre * gezinme=*liste;

 if(gezinme==NULL)//liste null ise zaten en kücük yok
   return;
 //liste bos değilse ilk elemani aliyorum,onunla diger elemanlari kiyaslayacam
    enk=gezinme;
    while(gezinme->sonraki!=NULL)
    {
       onceki=gezinme;
       gezinme=gezinme->sonraki;

       if(gezinme->icerik<enk->icerik)
      {

          enk=gezinme;//kiyaslanan eleman kucukse onu enk ata
          enkucuktenonceki=onceki;//en kucuk elemandan bir oncesini sakliyorum
       }
    }
    if(enk==*liste)//En kucuk eleman liste basinda ise liste basi degisecek
    {
       *liste=(*liste)->sonraki;

    }
    else//En kucuk eleman liste basinda degilse en kucuk elemanin oncesinde bulunan ile en kucuk
elemandan sonrakini birlestir.
    {
       enkucuktenonceki->sonraki=enk->sonraki;

    }
   free(enk);//En kucuk elemani siliyorum
}
```

```
//İki hash tablosunun içeriklerini birbirleriyle mukayese eden alt program farklı boyutlarda olsa bile
int aynimi(struct hash_tablosu *t1, struct hash_tablosu *t2){
if(t1==NULL&&t2==NULL) return 1;
if(t1!=NULL&&t2!=NULL)
{
int t1eleman=0,t2eleman=0,eslesneleman=0;
int i,j,k,l;
for(i=0;i<t1->tablo_uzunlugu;i++)
{
t1eleman+=t1->tablo_basi[i].counter;
}
for(i=0;i<t2->tablo_uzunlugu;i++)
{
t2eleman+=t2->tablo_basi[i].counter;
}
if(t1eleman!=t2eleman) return 0;
for(i=0;i<t1->tablo_uzunlugu;i++)
{
if(t1->tablo_basi[i].counter>0)
{
struct CELL *x=t1->tablo_basi[i].header;
while(x!=NULL)
{
for(j=0;j<t2->tablo_uzunlugu;j++)
{
if(t2->tablo_basi[j].counter>0)
{
struct CELL *y=t2->tablo_basi[j].header;
while(y!=NULL)
{
if(!strcmp(x->anahtar,y->anahtar))
{
eslesneleman++;
}
y=y->next;
}
}
}
x=x->next;
}
}
}
if(eslesneleman==t1eleman)
{
return 1;
}
else
{
return 0;
}
}
else return 0;
}
```